Module 2 — Workflows & Best Practices

Lesson 2.3

Review and refactor

Use Claude as a second pair of eyes on diffs, and as a careful executor of behavior-preserving change.

Why it matters

Review and refactor are where Claude's strengths (reading lots of code fast, spotting patterns, applying mechanical changes consistently) line up with where you most want a second opinion (you're tired, you wrote it, you're biased).

Two distinct activities

Activity	Goal	Risk
Review	Find issues before they ship	False negatives (missed bugs)
Refactor	Change shape without changing behavior	False positives (changed behavior)

Different prompts, different patterns.

Review patterns

Self-review your own diff

> *"Review the staged changes. I'm particularly worried about the migration's behavior under concurrent writes. What did I miss?"*

Always tell Claude *what you're worried about*. Generic "review this" gets a generic checklist.

Use the built-in /review skill

`/review` is the standard entry point — it inspects the current PR or pending changes and writes a structured review. It's tuned for the common review concerns (correctness, security, style consistency).

For security-sensitive changes: `/security-review`.

Independent second opinion via subagent

When you've been deep in a change and want a fresh perspective:

```
"Spawn a code-reviewer subagent. Brief: I just refactored the auth middleware.
Concern: backward compatibility with the old session token format.
Give me a second opinion — independent of my reasoning. Under 300 words."
```

The subagent has no exposure to your reasoning, so its read is genuinely independent. This is the right tool when you suspect you've talked yourself into a bad decision.

What review prompts should always include

- The diff or PR (Claude can read it via `git diff` or `gh pr view`)
- Your specific concerns
- The risk profile ("this hits prod tomorrow morning" vs. "low-stakes internal tool")
- The expected output shape ("a punch list, not an essay")

Refactor patterns

The behavior-preservation contract

> *"Refactor `UserService` to split out the email-sending logic into its own class.* ***Behavior must be unchanged.*** *All existing tests must still pass without modification. If you have to modify a test, stop and explain why."*

That last clause is critical. It catches the case where the refactor isn't actually behavior-preserving.

Refactor in tested territory only

If the code you want to refactor has no tests, you have two options:

1. Write tests first (yes, even if the goal is just to refactor). This pins behavior.
2. Refactor at your own risk.

Option 1 sounds slow but is faster than the rollback when option 2 breaks prod.

Mechanical refactors that Claude is great at

- Rename a symbol across the codebase
- Replace a deprecated API call with the new one
- Extract a constant or helper
- Move a function between files and fix all imports
- Convert callbacks to async/await (or sync to async)

Refactors to delegate to a subagent

Long, repetitive, file-by-file refactors burn context fast. Spawn a worker subagent per file or per directory:

```
"Use 3 parallel agents to migrate each of these three subdirectories from
Mocha to Vitest. Each agent owns one dir. Report file count migrated."
```

See Module 4 for orchestration patterns.

Try it

1. Make a small change in a branch. Don't commit.
2. Run `/review`.
3. Spawn an independent code-reviewer subagent with the brief above.
4. Compare the two outputs. Where do they overlap? Where do they differ? Which caught more?

Gotchas

- **"Refactor for clarity"** is a license to break things. Always pair with "behavior unchanged; tests pass."
- **Claude will rewrite passing tests** if not told otherwise. Be explicit.
- **Big-bang refactors fail.** Slice into independently reviewable, independently revertable PRs.

- **Review fatigue is real.** If `/review` returns a 40-item list, prioritize ruthlessly — the top 3 issues matter; the rest are noise.

Further reading

- 4.1 Subagents overview — independent reviewer pattern
- 2.5 Git and PRs — getting the diff into Claude's hands

Next

→ 2.4 Debugging patterns